

Introduction

Quiz Management & Online Assessment Platform

1. Introduction

The **Quiz Management & Online Assessment Platform** is a web-based application that allows students or users to participate in online quizzes and assessments.

The platform will have **two roles: Admin and Student/User**.

The **Admin** will have complete control over the platform, including creating quizzes, managing questions, managing users, monitoring quiz attempts, and viewing overall performance.

The **Student/User** will be able to browse available quizzes, attempt quizzes, view results, review answers, and track their performance.

The project is designed to provide practical experience with **frontend development, backend development, authentication, authorization, CRUD operations, database management, REST APIs, timers, scoring systems, dashboards, and responsive web design**.

2. Project Objectives

The main objectives are:

- Build a complete online quiz platform.
- Implement secure user authentication.
- Implement Admin and Student roles.
- Allow Admin to create and manage quizzes.
- Allow Admin to create and manage questions.
- Allow students to attempt quizzes online.
- Implement automatic scoring.
- Implement a countdown timer.
- Store quiz attempts and results.
- Provide performance analytics.
- Provide a leaderboard.
- Create separate Admin and Student dashboards.
- Build a responsive web application.

User Roles

3. User Roles

The application will have only **two roles**.

3.1 Admin

The Admin has complete control over the platform.

Admin Features

- Admin login
 - Admin dashboard
 - Manage students/users
 - Create quizzes
 - Edit quizzes
 - Delete quizzes
 - Publish/unpublish quizzes
 - Create questions
 - Edit questions
 - Delete questions
 - Manage categories
 - Manage difficulty levels
 - View all quiz attempts
 - View individual student results
 - View quiz performance
 - View platform analytics
 - Manage leaderboard
 - Activate/deactivate users
-

3.2 Student/User

Students can participate in quizzes available on the platform.

Student Features

- Register
- Login
- Logout
- View available quizzes

- Search quizzes
 - Filter quizzes
 - View quiz details
 - Start quiz
 - Answer questions
 - Navigate between questions
 - View remaining time
 - Submit quiz
 - View result
 - Review answers
 - View previous attempts
 - Track performance
 - View leaderboard
-

Main Application Modules

4. Main Application Modules

Module 1: Authentication

The application should provide authentication for Admin and Students.

Student Authentication

- Registration
- Login
- Logout
- Forgot password
- Reset password

Admin Authentication

- Admin login
- Logout
- Password reset

Admin accounts can either be created manually by the system owner or through a secure admin-management mechanism.

5. Admin Dashboard

The Admin Dashboard will be the central control panel of the application.

Dashboard Statistics

The Admin should be able to see:

- Total students
- Total quizzes
- Published quizzes
- Draft quizzes
- Total questions
- Total quiz attempts
- Average score

- Total passed attempts
- Total failed attempts

Dashboard Analytics

Display charts for:

- Quiz attempts over time
 - Student registrations
 - Average quiz scores
 - Pass/fail ratio
 - Most popular quizzes
 - Most popular categories
-

6. User Management

The Admin can manage all registered students.

Features

- View all students
- Search students
- View student profile
- View student's quiz history
- View student's performance
- Activate/deactivate account
- Delete account

Student Information

Name

Email

Registration Date

Account Status

Quizzes Attempted

Average Score

Highest Score

7. Quiz Management

All quiz management will be performed by the Admin.

Create Quiz

Admin can create a quiz containing:

- Quiz title
- Description
- Category
- Difficulty
- Duration
- Passing percentage
- Maximum attempts
- Quiz status
- Thumbnail/image (optional)

Example:

Quiz: JavaScript Fundamentals

Category: JavaScript

Difficulty: Intermediate

Duration: 20 minutes

Passing Score: 60%

Maximum Attempts: 2

Status: Published

Quiz Status

A quiz can have:

- Draft
- Published
- Unpublished

Only **published quizzes** should be available to students.

8. Category Management

Admin can manage quiz categories.

Examples:

- HTML
- CSS
- JavaScript
- React
- Node.js
- Python
- Java
- Database
- Computer Networks
- Cyber Security

Admin Features

- Create category
 - Edit category
 - Delete category
 - View quizzes under category
-

9. Question Management

Admin can manage questions for every quiz.

Question Information

Each question can contain:

- Question text
- Options
- Correct answer
- Explanation
- Marks
- Difficulty

Example:

Question

Which method converts a JSON string into a JavaScript object?

- A. JSON.stringify()
- B. JSON.parse()
- C. JSON.convert()
- D. JSON.object()

Correct Answer: JSON.parse()

Explanation: JSON.parse() converts a JSON string into a JavaScript object.

10. Question Types

The initial version can support:

Multiple Choice Question

One correct answer.

What does HTML stand for?

- Hyper Text Markup Language
- High Text Machine Language
- Hyper Tool Multi Language
- Home Tool Markup Language

Future Enhancement

The platform can later support:

- Multiple correct answers
- True/False
- Fill in the blanks
- Match the following
- Image-based questions
- Code-based questions

11. Quiz Attempt System

Students can start any published quiz.

The quiz interface should display:

JavaScript Fundamentals

Time Remaining: 14:32

Question 5 of 20

Which keyword is used to declare a constant?

- ☐ var
- ☐ let
- ☐ const
- ☐ static

[\[Previous\]](#) [\[Next\]](#)

[\[Submit Quiz\]](#)

The student should be able to:

- Select an answer
- Move to the next question
- Move to the previous question
- Navigate directly to a question
- See answered/unanswered questions
- Submit the quiz

12. Timer System

Every quiz can have a predefined duration.

Examples:

- 10 minutes
- 20 minutes
- 30 minutes
- 60 minutes

The timer should:

- Start when the quiz starts.
- Display remaining time.
- Continue correctly after page refresh where possible.
- Automatically submit the quiz when the time expires.

The actual quiz start time and expiry time should be validated by the backend rather than relying only on the browser timer.

13. Quiz Submission

When the student submits the quiz, the backend should process the submission.

The system should calculate:

- Correct answers
- Incorrect answers
- Unanswered questions
- Total marks
- Obtained marks
- Percentage
- Pass/fail status
- Time taken

Example:

Total Questions: 20

Correct Answers: 16

Incorrect Answers: 3

Unanswered: 1

Score: 80%

Status: PASSED

The scoring calculation should happen on the **backend** so students cannot manipulate their score through frontend code.

14. Result System

After completing a quiz, students should receive a detailed result.

Result Page

QUIZ RESULT

JavaScript Fundamentals

Score: 80%

Correct: 16

Incorrect: 3

Unanswered: 1

Time Taken: 14:21

Status: PASSED

Students should also be able to review:

- Question
 - Selected answer
 - Correct answer
 - Explanation
 - Whether the answer was correct
-

15. Student Dashboard

The Student Dashboard should provide an overview of the student's quiz activity.

Statistics

- Total quizzes attempted
- Total quizzes passed
- Total quizzes failed
- Average score
- Highest score
- Total questions answered

Dashboard Sections

Student Dashboard

Quizzes Attempted	15
Average Score	78%
Highest Score	96%
Passed	12
Failed	3

Recent Attempts

JavaScript Quiz	86%
React Quiz	92%
Python Quiz	74%

16. Quiz Discovery

Students should be able to find quizzes easily.

Search

Search by:

- Quiz title
- Category

Filters

Filter by:

- Category
 - Difficulty
 - Duration
 - Recently added
 - Popularity
-

17. Quiz Details Page

Before starting a quiz, students should see:

JavaScript Fundamentals

Description:

Test your knowledge of JavaScript fundamentals.

Category: JavaScript

Difficulty: Intermediate

Questions: 30

Duration: 30 Minutes

Passing Score: 60%

Maximum Attempts: 2

[Start Quiz]

18. Quiz Attempt History

Students should be able to view their previous attempts.

Example:

Quiz	Date	Score	Status
JavaScript	04 Aug	86%	Passed
React	02 Aug	72%	Passed
Python	30 Jul	48%	Failed

Clicking an attempt should open its detailed result.

19. Leaderboard

The platform can provide a leaderboard to increase engagement.

Rank students based on:

- Highest score
- Average score
- Number of quizzes completed

Example:

Rank	Student	Average Score
1	Rahul	96%
2	Priya	93%
3	Amit	91%

The leaderboard can be:

- Overall
 - Category-wise
 - Monthly
 - Weekly
-

20. Database Design

The project can use PostgreSQL/MySQL.

Users

users

id
name
email
password
role
status
created_at

The **role** field will contain:

ADMIN
STUDENT

Categories

categories

id
name
description
created_at

Quizzes

quizzes

id
title
description
category_id
difficulty
duration
passing_score
max_attempts
status
created_at
updated_at

Questions

questions

id
quiz_id
question_text
marks
explanation
difficulty
created_at

Options

options

id
question_id
option_text
is_correct

Attempts

attempts

id
quiz_id

user_id
score
percentage
correct_answers
incorrect_answers
unanswered
time_taken
status
started_at
completed_at

Answers

answers

id
attempt_id
question_id
selected_option_id
is_correct

Recommended Tech Stack

21. Recommended Tech Stack

Frontend

Recommended

- React.js
- Tailwind CSS

Additional Libraries

- React Router
 - Axios
 - Recharts
 - React Hook Form
-

Backend

The project can be implemented using:

- Python
 - Flask / Fast API / Django
-

22. Database

Recommended:

PostgreSQL

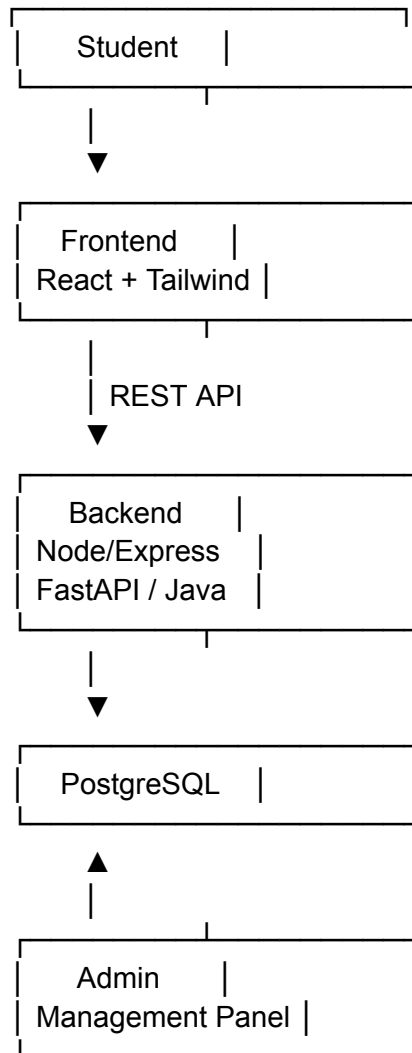
Alternative:

- MySQL
- MongoDB

PostgreSQL is recommended because the application has clear relationships between:

Users
↓
Attempts
↓
Quizzes
↓
Questions
↓
Options

23. Application Architecture



Two-Week Development Schedule

24. Two-Week Development Schedule

Week 1

Day 1 – Project Setup

- Frontend setup
- Backend setup
- Database setup
- Git repository
- Environment configuration

Day 2 – Authentication

- Student registration
- Login
- Logout
- Password hashing
- JWT/session authentication

Day 3 – Role-Based Authorization

- Admin role
- Student role
- Protected routes
- Admin middleware
- Student middleware

Day 4 – Admin Dashboard

- Dashboard layout
- Statistics
- Navigation
- User management

Day 5 – Quiz Management

- Create quiz
- Edit quiz
- Delete quiz
- Publish/unpublish quiz

Day 6 – Category & Question Management

- Category CRUD
- Add questions
- Add options
- Correct answer
- Question editing/deletion

Day 7 – Student Quiz Interface

- Quiz listing
 - Quiz details
 - Start quiz
 - Question navigation
 - Answer selection
 - Timer
-

Week 2

Day 8 – Quiz Submission

- Submit quiz
- Automatic submission
- Score calculation
- Pass/fail calculation

Day 9 – Results

- Result page
- Answer review
- Correct/incorrect answers
- Explanations
- Attempt history

Day 10 – Student Dashboard

- Statistics
- Quiz history
- Average score
- Performance charts

Day 11 – Admin Analytics

- Student statistics
- Quiz statistics
- Attempt statistics
- Pass/fail analytics

Day 12 – Leaderboard

- Ranking system
- Overall leaderboard
- Category leaderboard

Day 13 – Testing & Security

Test:

- Authentication
- Authorization
- Quiz creation
- Quiz attempts
- Timer
- Score calculation
- API validation
- Unauthorized access
- Input validation

Day 14 – Deployment & Documentation

- Deploy frontend
 - Deploy backend
 - Configure production database
 - Environment variables
 - Production testing
 - README
 - Screenshots
 - Project presentation
-

Other info and Conclusion

25. Important API Endpoints

Authentication

POST /api/auth/register
POST /api/auth/login
POST /api/auth/logout
POST /api/auth/forgot-password
POST /api/auth/reset-password

Users – Admin Only

GET /api/users
GET /api/users/:id
PUT /api/users/:id
DELETE /api/users/:id
PATCH /api/users/:id/status

Categories – Admin Only

GET /api/categories
POST /api/categories
PUT /api/categories/:id
DELETE /api/categories/:id

Quizzes

GET /api/quizzes
GET /api/quizzes/:id

POST /api/quizzes
PUT /api/quizzes/:id
DELETE /api/quizzes/:id

PATCH /api/quizzes/:id/publish

POST, PUT, DELETE, and publishing operations should be restricted to Admin.

Questions – Admin Only

GET /api/quizzes/:quizId/questions
POST /api/quizzes/:quizId/questions
PUT /api/questions/:id
DELETE /api/questions/:id

Quiz Attempts – Student

POST /api/quizzes/:quizId/start
POST /api/quizzes/:quizId/submit
GET /api/attempts
GET /api/attempts/:id

Admin Results

GET /api/admin/attempts
GET /api/admin/attempts/:id
GET /api/admin/analytics

Leaderboard

GET /api/leaderboard

26. Advanced Features

After completing the core requirements, students can implement additional features.

Question Randomization

Randomize questions for each attempt.

Option Randomization

Randomize answer options.

Negative Marking

Example:

Correct: +2

Wrong: -0.5

Skipped: 0

Maximum Attempts

Admin can configure the number of attempts allowed per quiz.

Quiz Scheduling

Admin can configure:

- Start date
- End date
- Availability

Certificate Generation

Generate a certificate when the student passes a quiz.

Email Notifications

Send emails for:

- Quiz completion
- Results
- Certificate generation

Dark Mode

Allow students and admins to switch between light and dark themes.

Question Import

Admin can upload questions using:

- CSV
- Excel

This can be a useful advanced feature for an internship-level project.

27. Security Requirements

The application should implement:

- Password hashing
- JWT/session security
- Role-based authorization
- Input validation
- SQL injection prevention
- XSS prevention
- CSRF protection where applicable
- Rate limiting
- Secure HTTP headers
- API validation
- Environment variable protection
- Secure error handling

Important Rule

The frontend must **never be trusted** for:

- Correct answers
- Scores
- User roles
- Quiz completion status
- Attempt eligibility

These must be validated by the backend.

28. Testing Requirements

Functional Testing

Test:

- Registration
- Login
- Admin login
- Quiz creation
- Question creation
- Quiz publishing
- Quiz attempt
- Quiz submission
- Score calculation
- Result generation

API Testing

Use:

- Postman
- Insomnia

Test:

- Valid requests
- Invalid requests
- Unauthorized requests
- Expired authentication
- Missing parameters
- Invalid IDs

Responsive Testing

The application should work properly on:

- Desktop
 - Laptop
-

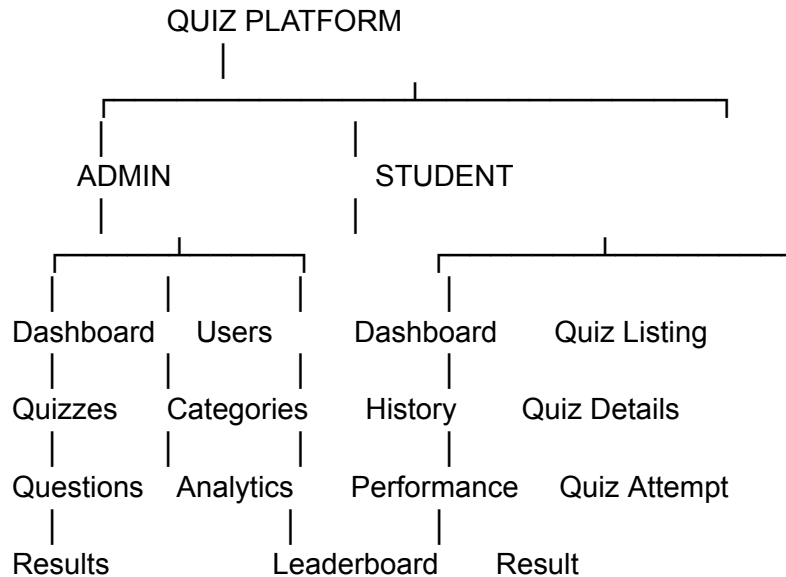
29. Learning Outcomes

After completing this project, students will understand:

- Frontend development
 - React.js
 - Backend development
 - REST APIs
 - Database design
 - PostgreSQL/MySQL
 - Authentication
 - Role-based authorization
 - CRUD operations
 - Form validation
 - State management
 - Timer implementation
 - Scoring algorithms
 - Dashboard development
 - Data visualization
 - API testing
 - Web security
 - Deployment
 - Git/GitHub
-

30. Expected Final Application

The completed application should contain:



31. Conclusion

The **Quiz Management & Online Assessment Platform** is a complete full-stack web development project with a simple and practical role structure.

There are only two roles:

Admin → Manages the entire platform

Student → Takes quizzes and tracks performance

The Admin is responsible for creating quizzes, managing questions and categories, managing students, publishing quizzes, and monitoring results. Students can discover quizzes, attempt them within a time limit, receive automatic results, review their answers, and monitor their performance.

The project provides students with practical experience in **authentication, authorization, CRUD operations, REST APIs, relational databases, timers, scoring systems, dashboards, analytics, and web security**, making it suitable for an **internship-level or portfolio full-stack project**.